

Implementação e Comparação de Algoritmos de Filtragem Colaborativa no Dataset MovieLens 100k

Pedro Henrique Ribeiro de Araujo Guedes

pedrohgues@ufrj.br

Universidade Federal Rural do Rio de Janeiro (UFRRJ)

Nova Iguaçu, RJ, Brasil

Resumo

Este trabalho apresenta a implementação, otimização e avaliação de dois algoritmos clássicos de Sistemas de Recomendação: um baseado em memória (*User-Based Collaborative Filtering*) e um baseado em modelo (*Regularized SVD*). Utilizando o conjunto de dados MovieLens 100k, as arquiteturas foram construídas de forma nativa com foco em álgebra linear vetorizada. Conduziu-se uma parametrização exaustiva para mitigar o viés de avaliação e controlar o fenômeno do *overfitting*. Adicionalmente, estabeleceu-se um *benchmarking* comparativo contra a biblioteca de referência *Scikit-Surprise*, analisando o compromisso entre precisão e eficiência computacional. Os resultados demonstram que a abordagem nativa atingiu o menor erro global ($RMSE = 0.9227$) neste cenário de teste, enquanto a biblioteca otimizada apresentou desempenho imensamente superior em escalabilidade e tempo de execução.

1 Introdução

Atualmente, há um crescimento exponencial no número de usuários que acessam sistemas de comércio eletrônico e plataformas de streaming de conteúdo. Estes sites oferecem um catálogo massivo de produtos e serviços, o que frequentemente sobrecarrega os consumidores nas suas decisões diárias de compra ou consumo. Neste cenário, os Sistemas de Recomendação (SR) têm desempenhado um papel crucial ao auxiliar os usuários a encontrar itens relevantes de forma personalizada [1].

Historicamente, grande parte das pesquisas em SR concentra-se no aprimoramento dos algoritmos de previsão para otimizar a acurácia [3]. A competição *Netflix Prize* demonstrou empiricamente que pequenas melhorias decimais na precisão geram impactos comerciais significativos [2]. Desde então, surgiram investigações robustas focadas em problemas como a esparsidade extrema dos dados e a personalização fina das recomendações, características que influenciam diretamente o desempenho dos recomendadores em ambientes de produção.

O objetivo deste trabalho é implementar e comparar diferentes algoritmos de filtragem colaborativa, incluindo abordagens baseadas em memória e baseadas em modelo, construídas de forma nativa a partir de rotinas vetorizadas. Explora-se a questão da hiperparametrização através de técnicas exaustivas de *Grid Search*, avaliando os resultados frente ao comportamento de uma biblioteca *open-source* amplamente utilizada na área.

2 Experimentos e Metodologia

2.1 Base de Dados e Protocolo de Avaliação

Os experimentos utilizaram a base de dados **MovieLens 100k**, composta por 100.000 avaliações distribuídas numa escala discreta de 1 a 5 estrelas. O *dataset* consolida interações de 943 usuários

sobre 1.682 filmes. Os dados brutos foram convertidos numa matriz esparsa de interações de formato 943×1682 .

Para garantir a robustez estatística e evitar o vazamento de dados (*data leakage*), utilizou-se a metodologia de partição *Holdout* com divisão de 80% das interações para o conjunto de treino e 20% para o conjunto de teste.

A qualidade das previsões foi mensurada por meio do Erro Quadrático Médio (*Root Mean Squared Error* - RMSE). O RMSE foi escolhido por penalizar erros grandes de previsão de forma mais severa, sendo amplamente utilizado na avaliação de sistemas de recomendação. Quanto menor o valor do RMSE, maior a proximidade entre as notas previstas e as notas reais. A métrica é expressa por:

$$RMSE = \sqrt{\frac{1}{|\mathcal{T}|} \sum_{(u,i) \in \mathcal{T}} (r_{u,i} - \hat{r}_{u,i})^2}$$

Onde $\mathcal{T}$ representa o conjunto de teste, $r_{u,i}$ a nota real e $\hat{r}_{u,i}$ a nota prevista pelo sistema.

2.2 Algoritmo Baseado em Memória: User-Based

A abordagem baseada em memória foi implementada utilizando a estrutura de vizinhança entre usuários. Para neutralizar o viés humano de avaliação — dado que usuários distintos adotam régua de notas sistematicamente diferentes —, aplicou-se a Correlação de Pearson através da centralização da matriz pelas médias individuais. A função de predição incorpora esta correção:

$$\hat{r}_{u,i} = \bar{r}_u + \frac{\sum_{v \in N_i(u)} \text{sim}(u,v) \cdot (r_{v,i} - \bar{r}_v)}{\sum_{v \in N_i(u)} |\text{sim}(u,v)|}$$

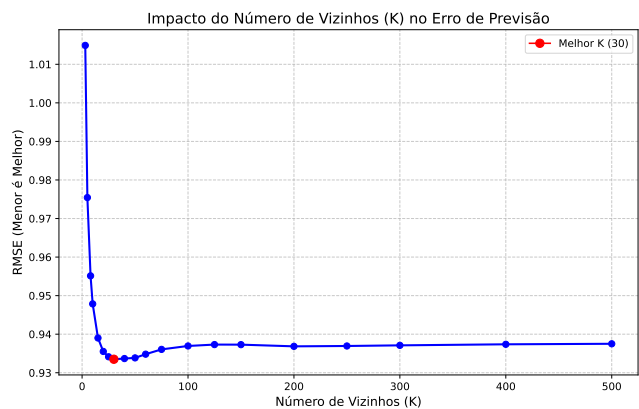


Figura 1: Impacto do Hiperparâmetro K (Vizinhos) no RMSE Global.

O comportamento do algoritmo perante a variação do número de vizinhos (K) é ilustrado na Figura 1. Um valor muito baixo de K incorre em alto erro devido à instabilidade induzida por preferências locais isoladas (*overfitting*). Com a expansão da vizinhança, o modelo estabiliza e atinge o seu ponto ótimo local em $K = 30$, registrando um **RMSE de 0.9335**. A partir deste limiar, a inclusão de vizinhos distantes insere ruído genérico, degradando sutilmente a acurácia.

2.3 Algoritmo Baseado em Modelo: SVD Regularizado

A filtragem baseada em modelo foi desenvolvida através da Fatoração de Matrizes por Decomposição em Valores Singulares (SVD) Regularizada. O modelo projeta usuários e itens num espaço latente partilhado de dimensionalidade F , estimando a nota através do produto escalar dos vetores correspondentes:

$$\hat{r}_{u,i} = p_u^T q_i$$

O processo de aprendizagem utilizou o algoritmo de Descida de Gradiente Estocástico (SGD) operando sobre os elementos não nulos da matriz de treino. Aplicou-se a regularização de Tikhonov (λ) para conter o crescimento dos pesos e garantir a generalização do modelo.

A parametrização ideal foi determinada por uma estratégia de refinamento em duas etapas (*Coarse-to-Fine Grid Search*). A busca de hiperparâmetros indicou que o melhor desempenho foi obtido com **80 Fatores Latentes**, **Taxa de Aprendizagem (α) = 0.01** e **Regularização (λ) = 0.02**, estabelecendo um erro mínimo de **0.9227**.

2.4 Análise Comparativa: Implementação Nativa vs. Scikit-Surprise

Para contextualizar a qualidade e a eficiência das estruturas matemáticas construídas, realizou-se um *benchmarking* comparativo sob as mesmas condições de partição contra a biblioteca *open-source* amplamente utilizada *Scikit-Surprise*. Os resultados quantitativos estão consolidados na Tabela 1.

Tabela 1: Comparação de Desempenho e Eficiência Computacional

Modelo / Arquitetura	RMSE Final	Tempo de Execução
Nativa: User-Based ($K = 30$)	0.9335	~ 2.10 s
Surprise: User-Based ($K = 30$)	1.0173	2.66 s
Nativa: SVD Regularizado	0.9227	~ 6.00 min
Surprise: SVD Regularizado	0.9599	0.82 s

A avaliação da Tabela 1 expõe um evidente compromisso entre precisão e tempo de execução. A abordagem nativa apresentou menor RMSE absoluto em ambos os cenários. Embora as bibliotecas otimizadas costumem obter resultados iguais ou superiores, o desempenho favorável da implementação manual neste experimento isolado pode ser justificado pelas condições restritas de teste. A sintonia fina (*Grid Search*) foi ajustada especificamente para a

implementação nativa e para a exata divisão do particionamento *Holdout* adotado. Ao aplicar estritamente os mesmos hiperparâmetros no *Surprise* — que utiliza uma arquitetura base diferente, incorporando *biases* de usuário e item por padrão —, é provável que o modelo da biblioteca tenha operado fora de sua configuração ideal, prejudicando a sua acurácia direta na comparação.

Por outro lado, o *Surprise* demonstrou uma vantagem substancial em eficiência de tempo no modelo SVD, concluindo o processamento em menos de um segundo. Este comportamento é justificado pelo *backend* da biblioteca, que delega os laços iterativos pesados para rotinas compiladas ao nível de máquina através de Cython (C-Extensions), mitigando o estrangulamento de interpretação nativo do Python que desacelerou o SGD implementado manualmente.

A Figura 2 estende a análise para o domínio comportamental dos modelos através de mapeamentos gráficos. No espectro da regressão (gráficos de dispersão), ambas as arquiteturas exibem o fenômeno de reversão à média, imposto pelas restrições de regularização. Os modelos adotam uma postura preditiva conservadora, atenuando a execução de notas limítrofes (1 e 5) para conter desvios drásticos.

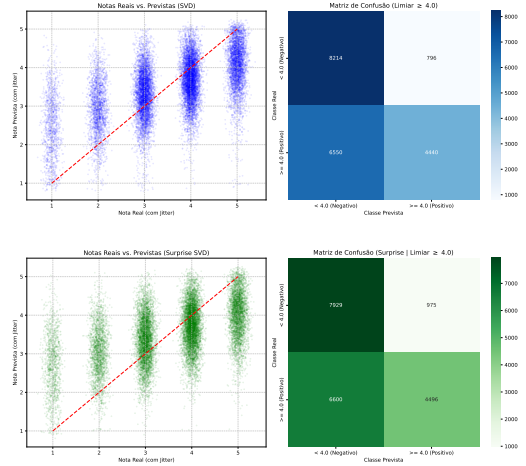


Figura 2: Gráficos de Dispersão e Matrizes de Confusão binarizadas no limiar de relevância ≥ 4.0 para a Implementação Nativa (Topo) e para a biblioteca Scikit-Surprise (Base).

Ao binarizar o problema para simular uma tarefa de recomendação real (onde itens com nota ≥ 4.0 são considerados recomendações corretas), as matrizes de confusão validam as propriedades estatísticas observadas. A implementação nativa registrou **8.214 Verdadeiros Negativos** e apenas **796 Falsos Positivos**, superando o *Surprise*, que computou **7.929 Verdadeiros Negativos** e **975 Falsos Positivos**.

Isto indica que o algoritmo manual minimizou de forma mais eficiente as recomendações inadequadas que poderiam frustrar a experiência do usuário. Em contrapartida, ambos exibem um volume similar e elevado de Falsos Negativos (6.550 nativo vs. 6.600 Surprise), evidenciando que a tendência do SVD à regressão à média penaliza o índice de *Recall* em ambas as frentes.

3 Conclusão

Este trabalho cumpriu os objetivos propostos ao implementar e estender modelos de filtragem colaborativa a partir de estruturas fundamentais. A decomposição matricial via SVD provou-se matematicamente mais eficaz para lidar com os desafios de esparsidade do MovieLens 100k do que o agrupamento por vizinhança. O processo de hiperparametrização foi essencial para calibrar os modelos, permitindo que a solução nativa apresentasse menor RMSE nos experimentos realizados.

Como sugestão para trabalhos futuros e visando contornar o comportamento excessivamente conservador identificado nas matrizes de confusão, recomenda-se a evolução do SVD elementar para

uma formulação que incorpore vieses basais explícitos (μ, b_u, b_i) integrados à decomposição. Adicionalmente, mapeia-se a necessidade de portar os laços de treino do SGD para extensões compiladas em C ou estruturas baseadas em GPU para equalizar o tempo de processamento à escala industrial.

Referências

- [1] Gediminas Adomavicius and Alexander Tuzhilin. 2005. Toward the next generation of recommender systems: a survey of the state-of-the-art and possible extensions. *IEEE Transactions on Knowledge and Data Engineering* 17, 6 (2005), 734–749.
- [2] Yehuda Koren, Robert Bell, and Chris Volinsky. 2009. Matrix factorization techniques for recommender systems. *Computer* 42, 8 (2009), 30–37.
- [3] Francesco Ricci, Lior Rokach, Bracha Shapira, and Paul B Kantor. 2011. *Recommender systems handbook*. Springer.